# Music App

## A Modern Android Music Player Built with Jetpack Compose

Overview, Architecture, and Key Features

**Presenter:** Omkaar | **Date:** March 09, 2026

# Project Structure and Core Imports

## Package & Android Basics

Package: `com.example.mcabclasswork.Misc.musicPlayer`

`android.media.AudioAttributes`, `MediaMetadataRetriever`, `MediaPlayer`

`android.net.Uri` for file handling

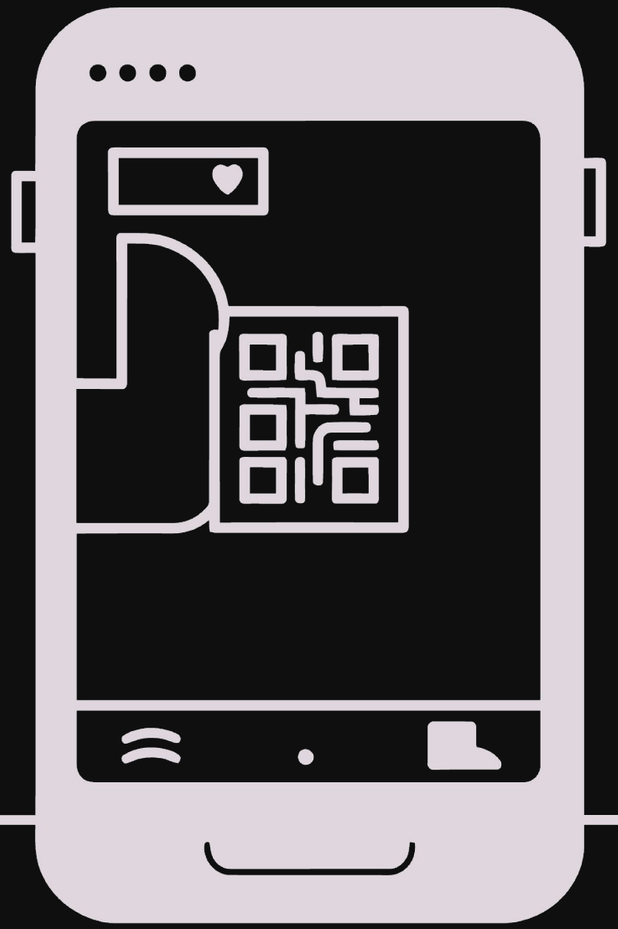
`ActivityResultContracts` for permissions

## Compose Activity Handling

`BackHandler` for navigation control

`rememberLauncherForActivityResult` for file picker

`MediaMetadataRetriever` extracts track metadata (title, artist, duration)



# Animation and UI Libraries

## Animation Libraries

`AnimatedContent`, `Crossfade`, `SharedTransitionLayout`

Easing: `FastOutLinearInEasing`, `FastOutSlowInEasing`

## Layout Components

`LazyColumn`, `Box`, `Column`, `Row`

Modifiers: `fillMaxSize`, `padding`, `statusBarsPadding`

## Visual Effects

`animatedColorAsState` for smooth color transitions

`infiniteRepeatable` for continuous animations

Heavy use of Compose animations enables fluid shared transitions between screens. Layouts create responsive, glassmorphic design.

# Theme Constants and Navigation

## Color Constants

```
val VoidBlack = Color(0xFF070709)
val GlassSurface = Color(0xFFFFFFFF).copy(alpha = 0.05f)
```

Dark, translucent glassmorphic theme with custom alpha values for translucent surfaces.

## Navigation Enum

```
enum class AppScreen { LIST, PLAYER, PROFILE }
```

Single-activity navigation between track browser, player, and profile screens.

## Graphics & Runtime

`CircleShape`, `RoundedCornerShape`
`Modifier.drawBehind`, `blur`, `clip`
`mutableStateOf`, `LaunchedEffect`
`rememberCoroutineScope` for async operations

# Data Handling: Tracks and Utilities

## 1

### Text Truncation Helper

```
fun truncateText(text:
String): String {
    val words = text.split("
").filter { it.isNotBlank() }
    return if (words.size > 2)
"${words[0]} ${words[1]}..."
else text
}
```

Limits display text to two words with ellipsis for clean UI.

## 2

### Track Data Class

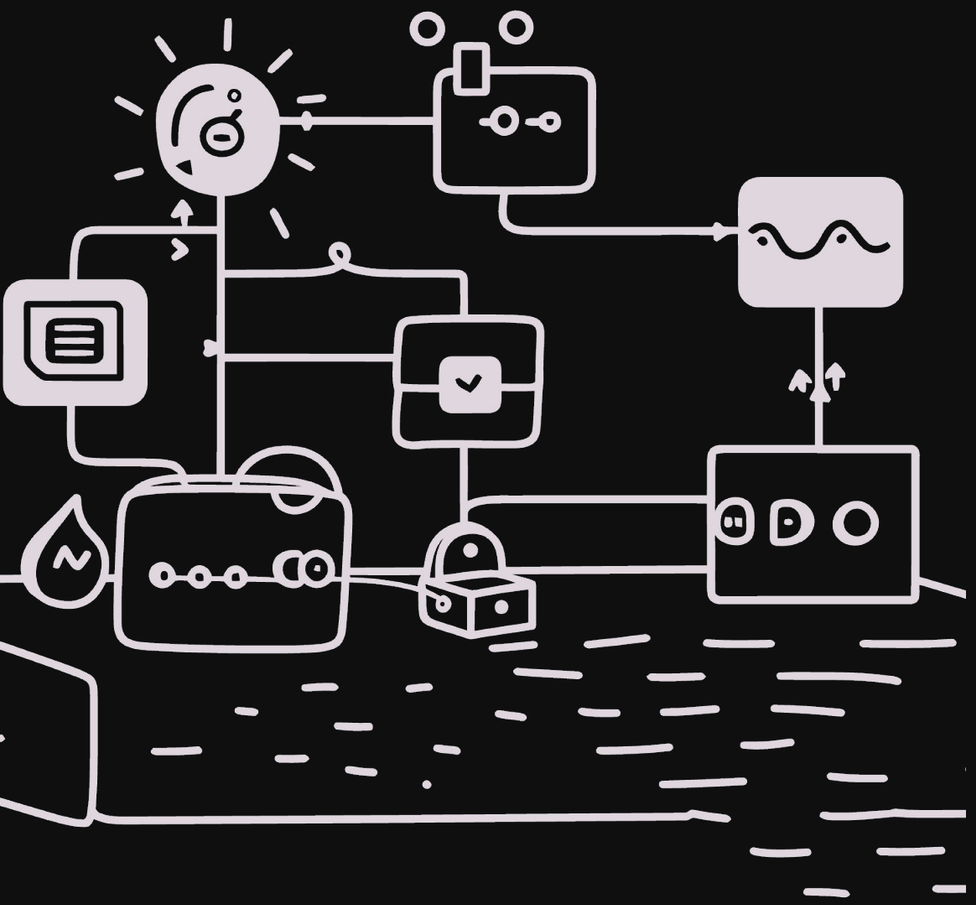
```
data class Track(
    val title: String, val
artist: String, val
durationMs: Long,
    val primaryColor: Color,
val primaryGradient: List,
    val accentGradient: List,
val logoStyle: Int,
    val contentUrl: Uri? =
null
)
```

Metadata plus bespoke "Harmonic Luminescence" colors and gradients.

## 3

### TrackStyle Generation

Predefined color palettes for 5 styles (Cyan, Pink, etc.). Styles cycle based on index for visual variety.



# Loading Tracks and Formatting

## getTrackFromUri Function

Extracts metadata using `MediaMetadataRetriever`:

- Sets source from URI
- Extracts title, artist, duration
- Applies generated style
- Releases retriever safely

## formatTimeMs Utility

```
fun formatTimeMs(ms: Long): String {
    val totalSeconds = ms / 1000
    val m = (totalSeconds / 60).toString().padStart(2, '0')
    val s = (totalSeconds % 60).toString().padStart(2, '0')
    return "$m:$s"
}
```

Ensures clean mm:ss display in UI.

# Reusable UI: ProfileAvatar

## Purpose

Canvas-drawn avatar for shared transitions with geometric shapes and futuristic look.

## Key Features

- Custom drawable with arcs and circles
- Optional notification dot
- Smooth animations
- Theme color customization

## Code Structure

```
@Composable
fun ProfileAvatar(themeColor: Color, showDot:
Boolean, modifier: Modifier = Modifier) {
 Box(modifier =
modifier.clip(CircleShape).background(GlassSurface).b
order(...)) {
 Canvas(modifier = Modifier.fillMaxSize()) {
 drawCircle(color = themeColor.copy(alpha = 0.2f),
radius = size.width / 2)
 drawCircle(color = themeColor, radius = size.width /
4, center = Offset(...))
 drawArc(color = themeColor.copy(alpha = 0.8f),
startAngle = 140f, sweepAngle = 260f, ...)
 }
 if (showDot) { /* Notification dot */ }
 }
}
```

# Navigation: GlassHeader Bar

1

## Top Navigation Bar

Glassmorphic background with avatar and add-music button for seamless navigation.

2

## Shared Transitions

Uses experimental `SharedTransitionLayout` for immersive screen transitions.

3

## File Picker Integration

Plus button launches file picker via `ActivityResultContracts` for adding tracks.

# Core Screens and Player Logic

### LIST Screen

Track browser with `LazyColumn` for efficient scrolling through local tracks.

### PLAYER Screen

Now playing with `MediaPlayer` controls, animated progress bars, and infinite Canvas visualizers.

### PROFILE Screen

User settings and app configuration with shared element transitions.

Screens use `rememberLazyListState` for state persistence and `animatedFloatAsState` for smooth progress animations.

# Summary: Features and Learnings

## Standout Features

- Glassmorphic UI with custom gradients
- Shared element animations
- Local track metadata extraction
- Coroutines for async loading

## Challenges Addressed

- URI permissions via contracts
- State management with mutableState
- Playback progress tracking

## Future Enhancements

- Full equalizer implementation
- Cloud sync capabilities
- Enhanced visualizers

This app showcases advanced Jetpack Compose techniques for modern Android development.